)

Politechnika
Wrocławska

Kurs: Procesory Osadzone ARM

Sprawozdanie

Projekt: Elektroniczna Kostka do Gry

Autor:
Kacper Sosnowski
Numer indeksu: 275800

Data: 26.06.2025 r.

Spis treści

1 Wprowadzenie **3**

2 Założenia Projektowe **3**
2.1 Funkcjonalności 3
2.2 Sposób Działania 3
2.3 Planowane Wykorzystanie Zasobów i Peryferiów 3

3 Zastosowane Komponenty i Sprzet **4**

4 Implementacja Techniczna **4**
4.1 Środowisko Deweloperskie 4
4.2 Konfiguracja Peryferiów 4
4.2.1 GPIO (General Purpose Input/Output) 4
4.2.2 TIM1 (Timer/PWM dla Buzzera) 5
4.2.3 RTC (Real-Time Clock) 5
4.3 Struktura Kodu i Główne Funkcje 5
4.3.1 Opis Zastosowanych Zmiennych 6
4.3.2 Wybrane Fragmenty Kodu 6
4.4 Diagram Blokowy Programu 10
4.5 Opis Połaczeń Sprzetowych 11
4.6 Szczegółowa Konfiguracja Peryferiów (STM32CubeIDE) 12
4.6.1 Konfiguracja Zegarów (System Clock Configuration) 12
4.6.2 Konfiguracja Pinów (Pinout View) 12

5 Podsumowanie i Wnioski **13**

1 Wprowadzenie

Projekt "Elektroniczna Kostka do Gry"stanowi implementacje cyfrowego odpowiednika tradycyjnej kostki do gier planszowych. Został on zrealizowany na mikrokontrolerze STM32F411E-DISCO. Głównym założeniem było stworzenie urzadzenia interaktywnego, które po naciśnieciu przycisku symuluje rzut kostka, prezentujac wynik w sposób wizualny i akustyczny. W trakcie tworzenia kodu, funkcjonalności zostały rozszerzone o animacje losowania, wzbogacajaca doświadczenia użytkownika.

2 Założenia Projektowe

Projekt bazował na poczatkowych założeniach, które w trakcie implementacji uległy modyfikacji w celu rozszerzenia funkcjonalności i uatrakcyjnienia urzadzenia.

2.1 Funkcjonalności

- **Losowanie liczby (1-6):** Główna funkcjonalność, uruchamiana po krótkim naciśnieciu przycisku użytkownika.
- **Wyświetlanie wyniku:** Wynik rzutu jest prezentowany na zewnetrznym 8-segmentowym wyświetlaczu. Oryginalne założenia wspominały również o diodach LED, które w zmodyfikowanej wersji zostały wykorzystane do animacji.
- **Sygnał dźwiekowy:** Buzzer emituje krótki dźwiek przy każdym rzucie. W rozwinietej wersji zaimplementowano bardziej złożone efekty dźwiekowe towarzyszace animacji.
- **Komunikacja UART:** Według założeń, wynik rzutu miał być wysyłany do komputera przez interfejs UART (np. jako tekst "Wylosowano: 3"). **Ta funkcjonalność nie została zaimplementowana w dostarczonym kodzie ze wzgledu na problemy z połaczeniem i komunikacja.**

2.2 Sposób Działania

Po naciśnieciu przycisku, system reaguje krótkim dźwiekiem poczatkowym z buzzera. Nastepnie rozpoczyna sie dynamiczna animacja losowania, podczas której:

- Na wyświetlaczu 8-segmentowym losowo i szybko zmieniaja sie cyfry (z zakresu 1-6).
- Wbudowane diody LED na płytce STM32F411E-DISCO (zielona, pomarańczowa, czerwona, niebieska) zapalaja sie sekwencyjnie.
- Buzzer emituje dźwiek o narastajacej czestotliwości, potegujac wrażenie "toczenia sie"kostki.

Po zakończeniu animacji (trwajacej około 1 sekundy), buzzer i diody LED zostaja wyłaczone. System losuje ostateczna liczbe z zakresu 1 do 6, która jest wyświetlana na wyświetlaczu 8-segmentowym. Cały proces kończy sie dłuższym, wyraźnym tonem buzzera, sygnalizujacym ostateczny wynik.

2.3 Planowane Wykorzystanie Zasobów i Peryferiów

- **Przycisk:** `USER_BUTTON` (PA0) – do wykrywania naciśniecia i inicjowania rzutu.
- **Buzzer:** Sterowany poprzez GPIO z wykorzystaniem sygnału PWM z timera (`TIM1`).

- **Wyświetlacz 8-segmentowy:** Do prezentacji wyniku, sterowany bezpośrednio z pinów GPIO mikrokontrolera.
- **Diody LED:** Wbudowane diody LED na płytce `STM32F411E-DISCO` (LD3, LD4, LD5, LD6 na porcie GPIOD), wykorzystane do animacji wizualnej.
- **UART:** `USART2` (PA2/PA3) – planowany do wysyłania wyniku rzutu do komputera. **Brak implementacji w kodzie ze wzgledu na problemy z połaczeniem i komunikacja.**
- **Generator liczb pseudolosowych:** Funkcja `rand()` biblioteki standardowej C, z **inicjalizacja ziarna** na podstawie czasu z RTC (`Real-Time Clock`) oraz `HAL_GetTick()`.

3 Zastosowane Komponenty i Sprzet

Do realizacji projektu wykorzystano nastepujace elementy:

- **Płytka Deweloperska:** STM32F411E-DISCO
- **Wyświetlacz 8-segmentowy:** Model FJ5461BH (Typu Common Anode).
- **Płytka Stykowa (Breadboard):** justPi.
- **Buzzer:** Buzzer z generatorem 5V 12mm - THT.
- **Przewody połaczeniowe:** Mesko-meskie oraz mesko-żeńskie.

4 Implementacja Techniczna

4.1 Środowisko Deweloperskie

Projekt został opracowany przy użyciu środowiska STM32CubeIDE, które umożliwia łatwa konfiguracje mikrokontrolerów STM32 oraz generowanie kodu z wykorzystaniem bibliotek `HAL (Hardware Abstraction Layer)`.

4.2 Konfiguracja Peryferiów

4.2.1 GPIO (General Purpose Input/Output)

- **Przycisk (`USER_BUTTON`):** Pin `PA0` został skonfigurowany jako wejście z zewnetrznym przerwaniem (`EXTI0`). Przerwanie jest wyzwalane zboczem narastajacym sygnału. W funkcji obsługi przerwania `HAL_GPIO_EXTI_Callback()` zastosowano prosty mechanizm debouncingu, aby zapobiec wielokrotnemu wykryciu jednego naciśniecia przycisku.
- **Wyświetlacz 8-segmentowy:** Segmenty (A-G) oraz kropka dziesietna (DP) zostały podłaczone do pinów portu D: `PD0` (`SEG_A_Pin`), `PD1` (`SEG_B_Pin`), `PD2` (`SEG_C_Pin`), `PD3` (`SEG_D_Pin`), `PD8` (`SEG_E_Pin`), `PD9` (`SEG_F_Pin`), `PD10` (`SEG_G_Pin`) oraz `PD11` (`SEG_DP_Pin`). Wszystkie te piny skonfigurowano jako wyjścia typu push-pull. Logika sterowania (niskie napiecie (`LOW`) dla właczenia segmentu, wysokie napiecie (`HIGH`) dla wyłaczenia) została skonfigurowana dla wyświetlacza typu Common Anode.
- **Diody LED (wbudowane):** Diody `LD4` (zielona - `PD12`), `LD3` (pomarańczowa - `PD13`), `LD5` (czerwona - `PD14`) i `LD6` (niebieska - `PD15`) na płytce Discovery również zostały skonfigurowane jako wyjścia typu push-pull.

4.2.2 TIM1 (Timer/PWM dla Buzzera)

`TIM1` został skonfigurowany do generowania sygnału PWM (Pulse Width Modulation) na kanale 1, sterujacym buzzerem. Czestotliwość zegara dla `TIM1` jest domyślnie równa czestotliwości `PCLK2`. Funkcja `buzzer_play_freq(uint16_t freq_hz, uint32_t duration_ms)` dynamicznie oblicza wartość rejestru `ARR (AutoReloadRegister)` oraz `CCR (Compare Capture Register)` w celu uzyskania zadanej czestotliwości i 50% wypełnienia sygnału. Dzieki temu możliwe jest generowanie tonów o różnej wysokości.

4.2.3 RTC (Real-Time Clock)

Moduł `RTC` został zainicjalizowany i wykorzystany do ziarnowania generatora liczb pseudolosowych (`rand()`). Odczyt aktualnych sekund, minut i godzin z RTC, w połaczeniu z wartościa `HAL_GetTick()`, zapewnia bardziej unikalne ziarno dla funkcji `srand()`, co z kolei przekłada sie na lepsza "losowość" generowanych liczb przy kolejnych uruchomieniach systemu.

4.3 Struktura Kodu i Główne Funkcje

- `displayDigit(int digit):` Główna funkcja odpowiedzialna za wyświetlanie zadanej cyfry (od 0 do 9 - testowe) na 8-segmentowym wyświetlaczu. Wykorzystuje stała tablice `segmentPatterns`, która definiuje, które segmenty powinny być właczone dla każdej cyfry. Funkcja odpowiednio steruje pinami GPIO.
- `setSegment(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin, GPIO_PinState state):` Pomocnicza funkcja do bezpośredniego ustawiania stanu logicznego (ON/OFF) pojedynczego segmentu wyświetlacza.
- `buzzer_play_freq(uint16_t freq_hz, uint32_t duration_ms):` Funkcja kontrolujaca buzzer. Aktywuje PWM na `TIM1`, ustawiajac żadana czestotliwość i utrzymujac sygnał przez określony czas, po czym automatycznie wyłacza buzzer.
- `buzzer_on()` / `buzzer_off():` Proste funkcje do manualnego właczania i wyłaczania buzzera, ustawiajace wypełnienie PWM na 50% lub 0%.
- `HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin):` Funkcja zwrotna biblioteki HAL, wywoływana automatycznie po wykryciu przerwania na pinie `PA0`. Ustawia globalna zmienna flagowa `buttonPressed` na `true`, sygnalizujac naciśniecie przycisku.
- `main():` Główna funkcja aplikacji.
  1. Inicjalizuje wszystkie peryferia mikrokontrolera.
  2. Pobiera dane z RTC i `HAL_GetTick()` do ziarnowania generatora liczb pseudolosowych.
  3. W nieskończonej petli `while(1)` nieustannie sprawdza stan flagi `buttonPressed`.
  4. Jeśli `buttonPressed` jest `true`:
     - Wyzerowuje flage `buttonPressed`.
     - Odtwarza poczatkowy krótki dźwiek buzzera.
     - Uruchamia petle animacji losowania trwajaca `ANIMATION_DURATION_MS` (1 sekunda). Wewnatrz tej petli:
       * Co `ANIMATION_STEP_DELAY_MS` (50 ms) wyświetlana jest nowa, losowa cyfra na wyświetlaczu 7-segmentowym.
       * Generowany jest krótki dźwiek buzzera o stopniowo wzrastajacej czestotliwości (`current_freq`).

    - Wbudowane diody LED na płytce Discovery zapalaja sie kolejno, tworzac efekt wizualny.
  - Po zakończeniu animacji, buzzer i wszystkie diody LED sa wyłaczane.
  - Losowana jest ostateczna liczba (`randomNumber`) z zakresu 1 do 6.
  - Ostateczny wynik jest wyświetlany na wyświetlaczu 8-segmentowym.
  - Odtwarzany jest końcowy, dłuższy i wyższy ton buzzera.
  - Krótka pauza (`HAL_Delay(200);`) przed ponownym wejściem w stan oczekiwania.

4.3.1 Opis Zastosowanych Zmiennych

W projekcie wykorzystano nastepujace zmienne i definicje, które odpowiadaja za sterowanie logika programu oraz parametrami działania peryferiów:

- `segmentPatterns[10][8]`: Stała tablica typu `uint8_t` przechowujaca wzorce bitowe dla wyświetlacza 8-segmentowego (Common Anode). Każdy wiersz odpowiada jednej cyfrze (od 0 do 9), a kolumny reprezentuja stan poszczególnych segmentów (A-G oraz DP). `LOW` oznacza właczenie segmentu, a `HIGH` wyłaczenie.
- `buttonPressed`: Zmienna globalna typu `volatile bool`, służaca jako flaga sygnalizujaca naciśniecie przycisku użytkownika. Jest zmieniana w funkcji obsługi przerwania `HAL_GPIO_EXTI_Callback()` i sprawdzana w głównej petli `main()`. Słowo kluczowe `volatile` zapewnia poprawna optymalizacje dostepu do zmiennej z poziomu przerwania i głównego watku.
- `animationRunning`: Zmienna globalna typu `volatile bool`, sygnalizujaca czy animacja rzutu kostka jest aktualnie w trakcie. Używana do koordynacji zachowania buzzera i diod LED podczas animacji.
- `ANIMATION_DURATION_MS`: Definicja (`#define`) określajaca całkowity czas trwania animacji losowania (domyślnie 1000 ms, czyli 1 sekunda).
- `ANIMATION_STEP_DELAY_MS`: Definicja (`#define`) określajaca opóźnienie (krok) miedzy kolejnymi zmianami wyświetlanych cyfr i stanów diod LED w trakcie animacji (domyślnie 50 ms).
- `BUZZER_DURATION_MS`: Definicja (`#define`) określajaca czas trwania krótkich dźwieków buzzera, np. dźwieku poczatkowego.
- `BUZZER_TICK_DURATION_MS`: Definicja (`#define`) określajaca czas trwania pojedynczego, krótkiego "kliku"buzzera podczas animacji (domyślnie 10 ms).
- `randomNumber`: Zmienna typu `int`, przechowujaca ostatecznie wylosowana liczbe (wynik rzutu kostka).
- `allLeds[]`: Tablica typu `uint16_t` zawierajaca piny GPIO wszystkich wbudowanych diod LED na płytce Discovery (LD4, LD3, LD5, LD6), używana do sekwencyjnego sterowania nimi podczas animacji.

4.3.2 Wybrane Fragmenty Kodu

Poniżej przedstawiono kluczowe fragmenty kodu źródłowego, które odpowiadaja za realizacje głównych funkcjonalności projektu. Pełny kod znajduje sie w załaczniku.

Funkcja `displayDigit(int digit)` Funkcja `displayDigit` jest odpowiedzialna za wyświetlanie cyfr na 8-segmentowym wyświetlaczu. Wykorzystuje globalna tablice `segmentPatterns` do określenia, które segmenty należy właczyć (ustawić stan `LOW`) dla danej cyfry.

```c
void displayDigit(int digit) {
    if (digit < 0 || digit > 9) {
        // Wszystkie segmenty OFF (HIGH) w przypadku niepoprawnej cyfry
        setSegment(SEG_A_GPIO_Port, SEG_A_Pin, HIGH);
        setSegment(SEG_B_GPIO_Port, SEG_B_Pin, HIGH);
        setSegment(SEG_C_GPIO_Port, SEG_C_Pin, HIGH);
        setSegment(SEG_D_GPIO_Port, SEG_D_Pin, HIGH);
        setSegment(SEG_E_GPIO_Port, SEG_E_Pin, HIGH);
        setSegment(SEG_F_GPIO_Port, SEG_F_Pin, HIGH);
        setSegment(SEG_G_GPIO_Port, SEG_G_Pin, HIGH);
        setSegment(SEG_DP_GPIO_Port, SEG_DP_Pin, HIGH);
        return;
    }

    // Ustawia piny segment w na podstawie tablicy segmentPatterns
    HAL_GPIO_WritePin(SEG_A_GPIO_Port, SEG_A_Pin, segmentPatterns[digit][0]); // PD0 (Segment A)
    HAL_GPIO_WritePin(SEG_B_GPIO_Port, SEG_B_Pin, segmentPatterns[digit][1]); // PD1 (Segment B)
    HAL_GPIO_WritePin(SEG_C_GPIO_Port, SEG_C_Pin, segmentPatterns[digit][2]); // PD2 (Segment C)
    HAL_GPIO_WritePin(SEG_D_GPIO_Port, SEG_D_Pin, segmentPatterns[digit][3]); // PD3 (Segment D)
    HAL_GPIO_WritePin(SEG_E_GPIO_Port, SEG_E_Pin, segmentPatterns[digit][4]); // PD8 (Segment E)
    HAL_GPIO_WritePin(SEG_F_GPIO_Port, SEG_F_Pin, segmentPatterns[digit][5]); // PD9 (Segment F)
    HAL_GPIO_WritePin(SEG_G_GPIO_Port, SEG_G_Pin, segmentPatterns[digit][6]); // PD10 (Segment G)
    HAL_GPIO_WritePin(SEG_DP_GPIO_Port, SEG_DP_Pin, segmentPatterns[digit][7]);// PD11 (Segment DP)
}
```

Listing 1: Funkcja displayDigit()

Funkcja `HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)` Ta funkcja jest callbackiem z biblioteki `HAL`, wywoływanym automatycznie po wykryciu zbocza narastajacego na pinie `PA0` (przycisk użytkownika). Implementuje prosty debouncing, aby zapobiec wielokrotnemu zarejestrowaniu jednego naciśniecia i ustawia flage `buttonPressed`.

```c
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    if (GPIO_Pin == GPIO_PIN_0) { // Sprawd , czy przerwanie pochodzi z PA0 (USER_BUTTON)
        // Proste debouncing (mo na rozbudowa  o timer)
        static uint32_t last_press_time = 0;
        uint32_t current_time = HAL_GetTick();

        if (current_time - last_press_time > 200) { // Odczekaj
```

```
            200ms na debouncing
            buttonPressed = true;
            last_press_time = current_time;
        }
    }
}
```

Listing 2: Funkcja HAL_GPIO_EXTI_Callback()

Fragment Petli Głównej `while(1)` **z** `main()` Poniższy fragment kodu z funkcji `main()` pokazuje logike działania aplikacji po wykryciu naciśniecia przycisku. Zawiera obsługe animacji, generowanie losowej liczby i sygnały dźwiekowe.

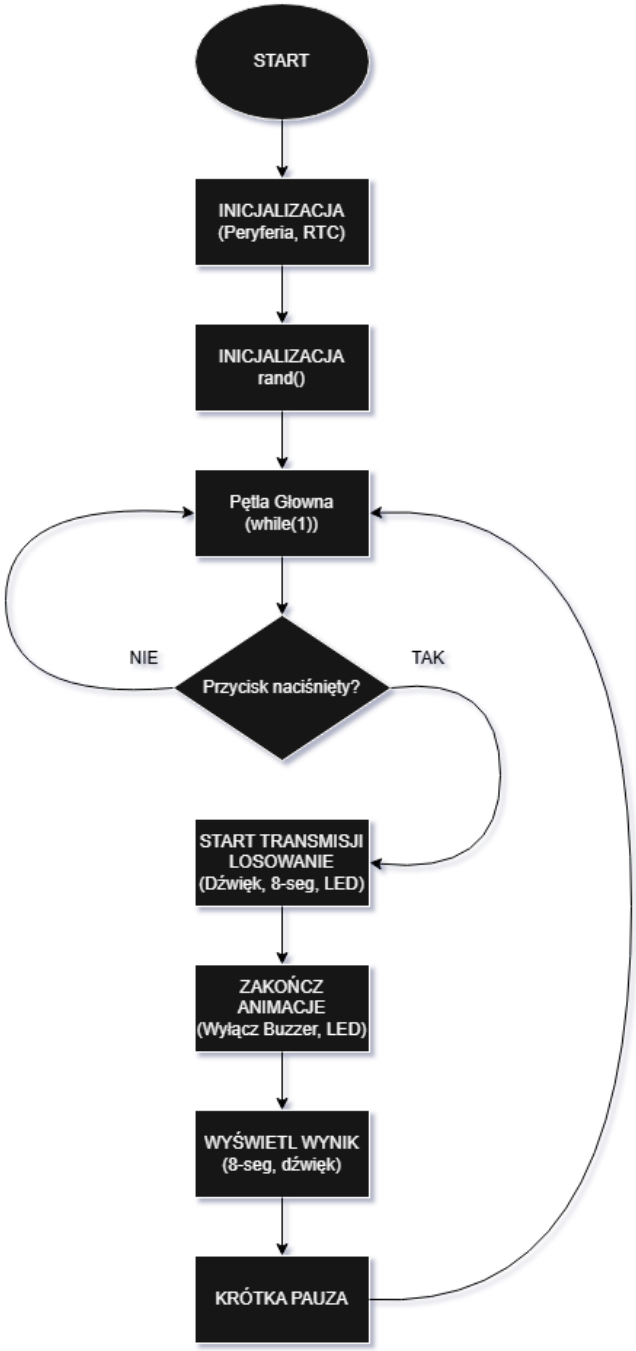
```
  while (1)
  {
      if (buttonPressed) {
            // Wyzeruj flag , aby unikn    wielokrotnego wykonania
            buttonPressed = false;

            // 1. D wi k pocz tkowy (kr tki "klik")
            buzzer_play_freq(1000, BUZZER_DURATION_MS);

            // 2. Rozpocz cie animacji losowania
            animationRunning = true;
            uint32_t animationStartTime = HAL_GetTick();
            uint32_t nextAnimationStepTime = animationStartTime;

            uint16_t current_freq = 300; // Cz stotliwo startowa buzzera
            uint16_t freq_step = 50;     // Krok zwi kszania cz stotliwo ci

            uint16_t allLeds[] = {LD4_Pin, LD3_Pin, LD5_Pin, LD6_Pin}; // Diody LED
            int currentLedIndex = 0;

            while (HAL_GetTick() - animationStartTime < ANIMATION_DURATION_MS) {
                if (HAL_GetTick() >= nextAnimationStepTime) {
                    nextAnimationStepTime += ANIMATION_STEP_DELAY_MS;

                    // Losuj i wy wietl losow  cyfr  (1-6) na potrzeby animacji
                    uint8_t current_animation_digit = (rand() % 6) + 1;
                    displayDigit(current_animation_digit);

                    // Buzzer: narastaj cy ton
                    buzzer_play_freq(current_freq,
```

```
                        BUZZER_TICK_DURATION_MS);
                    current_freq += freq_step;
                    if (current_freq > 2000) {
                        current_freq = 300;
                    }

                    // Diody LED: za wie  jedn  w k  ko
                    HAL_GPIO_WritePin(LED_GPIO_Port, LD4_Pin |
                        LD3_Pin | LD5_Pin | LD6_Pin, GPIO_PIN_SET);
                    HAL_GPIO_WritePin(LED_GPIO_Port, allLeds[
                        currentLedIndex], GPIO_PIN_RESET);
                    currentLedIndex = (currentLedIndex + 1) % 4;
                }
            }
            animationRunning = false;
            buzzer_off();
            HAL_GPIO_WritePin(LED_GPIO_Port, LD4_Pin | LD3_Pin |
                LD5_Pin | LD6_Pin, GPIO_PIN_SET);

            // 3. Po zako czeniu animacji, wylosuj ostateczn
                liczb  i wy wietl j
            randomNumber = (rand() % 6) + 1;
            displayDigit(randomNumber);

            // 4. D wi k ko cowy (np. d u szy, wy szy ton)
            buzzer_play_freq(1500, 150);
            buzzer_off();
            HAL_Delay(200);
        }
    }
```

Listing 3: Fragment petli głównej main()

4.4 Diagram Blokowy Programu

Poniższy diagram przedstawia uproszczony schemat blokowy działania programu "Elektroniczna Kostka do Gry". Skupia sie on na głównych procesach i przepływie sterowania.



Rysunek 1: Diagram blokowy działania programu Elektronicznej Kostki do Gry.

4.5 Opis Połaczeń Sprzetowych

Poniższy opis przedstawia sposób połaczenia mikrokontrolera STM32F411E-DISCO z zewnetrznymi peryferiami: wyświetlaczem 8-segmentowym oraz buzzerem. Pokazuje on również wykorzystanie wbudowanego przycisku USER_BUTTON oraz diod LED na płytce.

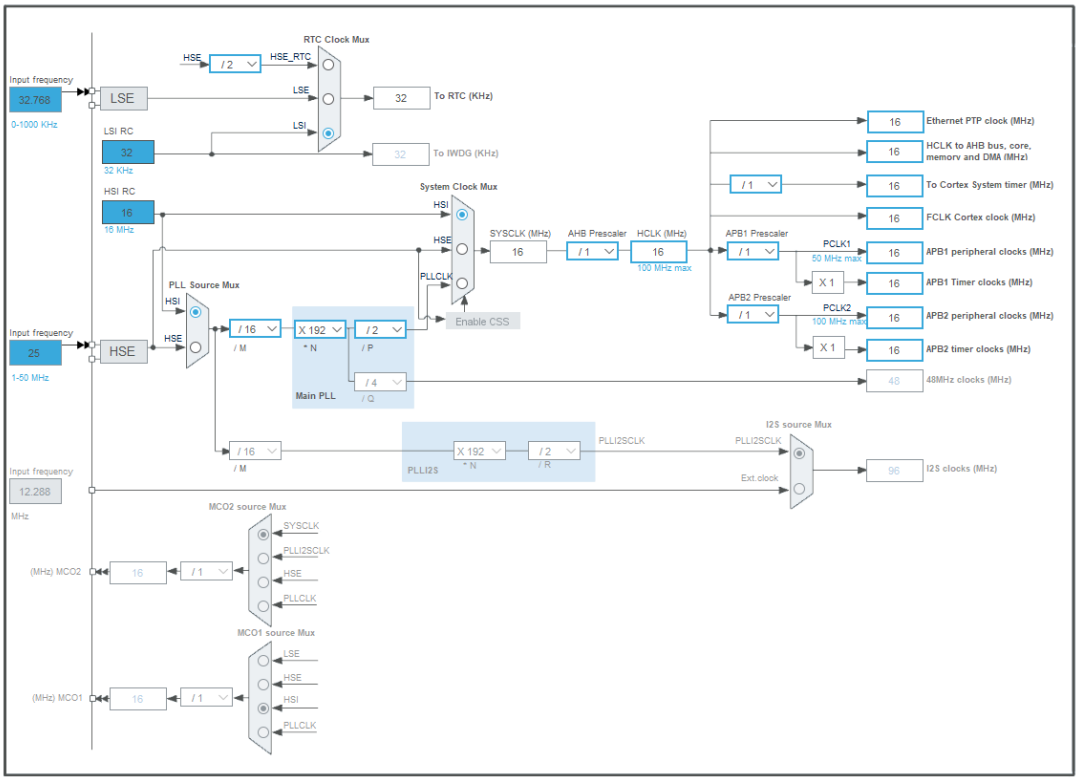
- **Wyświetlacz 8-segmentowy:**
  - Segment A: PD0
  - Segment B: PD1
  - Segment C: PD2
  - Segment D: PD3
  - Segment E: PD8
  - Segment F: PD9
  - Segment G: PD10
  - Segment DP (kropka): PD11
  - Zasilanie (Common Anode): Podłaczone do 3.3V
- **Buzzer pasywny:**
  - Sterowanie PWM: PA8 (TIM1 Channel 1)
  - Drugi pin buzzera: GND
- **Przycisk Użytkownika (`USER_BUTTON`):**
  - Pin: PA0 (Wbudowany na płytce Discovery, podłaczony do masy, a mikrokontroler z wewnetrznym pull-up lub zewnetrznym pull-down).
- **Diody LED (wbudowane na płytce Discovery):**
  - Zielona (LD4): PD12
  - Pomarańczowa (LD3): PD13
  - Czerwona (LD5): PD14
  - Niebieska (LD6): PD15

4.6 Szczegółowa Konfiguracja Peryferiów (STM32CubeIDE)

Poniżej przedstawiono zrzuty ekranu z konfiguracji projektu w środowisku STM32CubeIDE (CubeMX), ilustrujace ustawienia peryferiów mikrokontrolera STM32F411E-DISCO.

4.6.1 Konfiguracja Zegarów (System Clock Configuration)

W tej sekcji przedstawiono ustawienia zegarów mikrokontrolera, kluczowe dla prawidłowego taktowania wszystkich modułów.

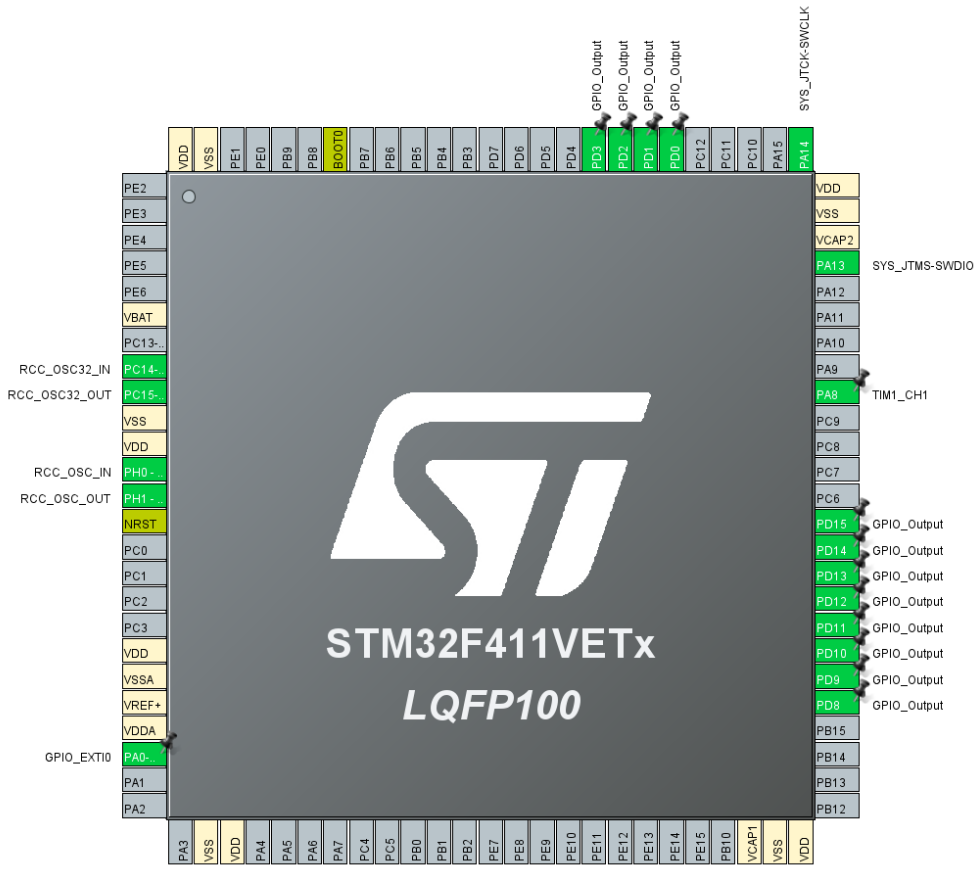


Rysunek 2: Zrzut ekranu z konfiguracji zegarów (Clock Configuration) w STM32CubeIDE.

W projekcie zastosowano wewnetrzny oscylator HSI o czestotliwości 16 MHz jako źródło zegara systemowego (SYSCLK). Czestotliwości magistral AHB, APB1 i APB2 zostały ustawione na 16 MHz, bez preskalerów (dzielnik 1). Oznacza to, że wszystkie peryferia taktowane sa z maksymalna czestotliwościa 16 MHz.

4.6.2 Konfiguracja Pinów (Pinout View)

W tej sekcji przedstawiono ogólny widok konfiguracji pinów mikrokontrolera, z zaznaczonymi przypietymi peryferiami i ich funkcjami.



Rysunek 3: Zrzut ekranu z widoku Pinout Configuration w STM32CubeIDE z zaznaczonymi konfiguracjami pinów.

Zrzut ekranu powyżej przedstawia graficzna reprezentacje konfiguracji wszystkich wykorzystywanych pinów. Wskazuje on, które piny zostały skonfigurowane jako wyjścia GPIO dla wyświetlacza 8-segmentowego i diod LED (GPIOD), wejście z przerwaniem dla przycisku użytkownika (PA0), oraz wyjście PWM dla buzzera (PA8).

5 Podsumowanie i Wnioski

Projekt "Elektroniczna Kostka do Gry"został z sukcesem zrealizowany na platformie STM32F411E-DISCO. Wprowadzone rozszerzenia, takie jak animacja losowania z dynamicznym dźwiekiem i sekwencyjnym miganiem diod LED, znaczaco podniosły interaktywność i atrakcyjność urzadzenia w porównaniu do bazowych założeń. Skutecznie wykorzystano podstawowe peryferia mikrokontrolera (GPIO, Timery w trybie PWM, RTC do ziarnowania generatora liczb pseudolosowych).

Jedynym odstepstwem od oryginalnych założeń jest brak implementacji komunikacji UART, która miała służyć do wysyłania wyniku rzutu do komputera. **Problemy z nawiazaniem stabilnego połaczenia i konfiguracji interfejsu UART uniemożliwiły jej pełne wdrożenie w założonym czasie.** Całość projektu stanowi jednak solidna podstawe i może być rozwijana w przyszłości o dodatkowe funkcjonalności, takie jak:

- Implementacja interfejsu UART.

- Wykorzystanie zewnetrznej pamieci (np. EEPROM) do przechowywania historii rzutów.
- Dodanie obsługi wielu kostek.
- Bardziej zaawansowane algorytmy generowania dźwieku lub wzorców świetlnych.
- Użycie innych interfejsów do komunikacji np. I2C/SPI do podłaczenia dodatkowych modułów.

Projekt dowodzi umiejetności programowania mikrokontrolerów oraz integracji różnych komponentów sprzetowych w celu realizacji zadanego celu.